# Autoevaluación

## Tabla de contenidos

1 Complejidad computacional ..... 1
2 Estructuras de datos lineales ..... 4

## 1 Complejidad computacional

1. Para un arreglo que contiene 100 elementos, indica la cantidad de pasos que llevarían las siguientes operaciones:

   a. Lectura
   b. Búsqueda de un valor que no está contenido en el arreglo
   c. Inserción al inicio del arreglo
   d. Inserción al final del arreglo
   e. Eliminación al inicio del arreglo
   f. Eliminación al final del arreglo

2. Para un conjunto basado en un arreglo que contiene 100 elementos, indica la cantidad de pasos que llevarían las siguientes operaciones:

   a. Lectura
   b. Búsqueda de un valor que no está contenido en el conjunto
   c. Inserción de un nuevo valor al inicio del conjunto
   d. Inserción de un nuevo valor al final del conjunto
   e. Eliminación al inicio del conjunto
   f. Eliminación al final del conjunto

3. En general, la operación de búsqueda en un arreglo busca la primera instancia de un valor dado. Pero a veces queremos buscar todas las instancias de un valor determinado. Por ejemplo, supongamos que queremos contar cuántas veces aparece el valor `"manzana"` dentro de un arreglo. ¿Cuántos pasos tomaría encontrar todas las apariciones de `"manzana"`? Da tu respuesta en función de N.

4. Use la notación *Big O* para describir la complejidad temporal de las siguientes funciones:

```python
def es_bisiesto(valor):
    if valor % 100 == 0:
        if valor % 400 == 0:
            return False
        else:
            return True
    return valor % 4 == 0
```

```python
def suma(arreglo):
    total = 0
    for numero in arreglo:
        total += numero
    return total
```

5. La siguiente función recibe un arreglo de cadenas y devuelve un nuevo arreglo que solo contiene las cadenas que comienzan con la letra `"a"`. Use la notación *Big O* para describir la complejidad temporal de la función:

```python
def seleccionar_cadenas_a(arreglo):
    arreglo_nuevo = []
    for cadena in arreglo:
        if cadena[0] == "a":
            arreglo_nuevo.append(cadena)
    return arreglo_nuevo
```

6. La siguiente función calcula la mediana de un arreglo ordenado. Describa su complejidad temporal en términos de la notación *Big O*:

```python
def mediana(arreglo):
    if not arreglo:
        return None

    medio = len(arreglo) // 2

    # Si el arreglo tiene una cantidad par de números:
    if len(arreglo) % 2 == 0:
        return (arreglo[medio - 1] + arreglo[medio]) / 2.0
    else:  # Si el arreglo tiene una cantidad impar de números:
        return arreglo[medio]
```

7. Reemplace los signos de interrogación en la siguiente tabla para describir cuántos pasos ocurren para una cantidad dada de elementos de datos, según distintos tipos de *Big O*:

| N elementos | O(N) | O(log N) | O($N^2$) |
|---|---|---|---|
| 100 | 100 | ? | ? |
| 2000 | ? | ? | ? |

8. La siguiente función encuentra el producto más grande posible entre cualquier par de números dentro de un arreglo dado. Usá la notación *Big O* para describir la complejidad temporal de la siguiente función:

```
def greatest_product(arreglo):
    if len(producto_mas_grande) < 2:
        return None

    producto_mas_grande = arreglo[0] * arreglo[1]

    for i, valor_i in enumerate(arreglo):
        for j, valor_j in enumerate(arreglo):
            if (i != j and valor_i * valor_j > producto_mas_grande):
                producto_mas_grande = valor_i * valor_j

    return producto_mas_grande
```

9. La siguiente función verifica si un arreglo de números contiene un par de números cuya suma sea igual a 10:

```
def suma_de_pares(arreglo):
    for i, valor_i in enumerate(arreglo):
        for j, valor_j in enumerate(arreglo):
            if (i != j) and (valor_i + valor_j == 10):
                return True
    return False
```

   a. ¿Cuál es el mejor caso, el caso promedio y el peor caso?
   b. Exprese el peor caso en términos de notación *Big O*.

10. La siguiente función encuentra el número más grande dentro de un arreglo, pero tiene una eficiencia de $O(N^2)$. Reescribí la función para que sea una versión más rápida, con eficiencia O(N):

```
def maximo(arreglo):
    if not arreglo:
        return None

    for i in arreglo:
        # Supongamos por ahora que i es el mayor
        es_i_el_mas_grande = True

        for j in arreglo:
            # Si encontramos otro valor mayor que i, i no es el mayor
            if j > i:
                es_i_el_mas_grande = False

        # Si, al revisar todos los otros números, i sigue siendo el mayor,
        # entonces i es el número más grande:
        if es_i_el_mas_grande:
            return i
```

## 2 Estructuras de datos lineales

1. Si estuvieras desarrollando un programa para un centro de llamadas que pone a los clientes en espera y luego los asigna al "próximo representante disponible", ¿usarías una pila o una cola?
2. Si se apilan números en una pila en el siguiente orden: 1, 2, 3, 4, 5, 6, y luego se desapilan dos elementos, ¿qué número quedaría visible en la parte superior de la pila?
3. Si se insertan números en una cola en el siguiente orden: 1, 2, 3, 4, 5, 6, y luego se extraen dos elementos, ¿qué número sería el siguiente en salir de la cola?
4. Escriba una función que use una pila para invertir una cadena de texto. (Por ejemplo, "abcde" debería convertirse en "edcba")
5. ¿Qué valores se devuelven durante la siguiente serie de operaciones sobre una pila, si se ejecutan sobre una pila inicialmente vacía?

```
insertar(5)
insertar(3)
extraer()
insertar(2)
insertar(8)
extraer()
extraer()
insertar(9)
insertar(1)
extraer()
insertar(7)
insertar(6)
extraer()
extraer()
insertar(4)
extraer()
extraer()
```

6. ¿Qué valores se devuelven durante la siguiente secuencia de operaciones sobre una cola, si se ejecutan sobre una cola inicialmente vacía?

```
insertar(5)
insertar(3)
extraer()
insertar(2)
insertar(8)
extraer()
extraer()
insertar(9)
insertar(1)
extraer()
```

```
insertar(7)
insertar(6)
extraer()
extraer()
insertar(4)
extraer()
extraer()
```

7. Suponga que se realiza una secuencia de operaciones de pila (`insertar` y `extraer`). Las operaciones de inserción colocan los enteros del 0 al 9, en orden, dentro de la pila; las operaciones extracción imprimen los valores que se extraen. ¿Cuál o cuáles de las siguientes secuencias no podrían ocurrir?

   a. `4 3 2 1 0 9 8 7 6 5`
   b. `4 6 8 7 5 3 2 9 0 1`
   c. `2 5 6 7 4 8 9 3 1 0`
   d. `4 3 2 1 0 5 6 7 8 9`
   e. `1 2 3 4 5 6 9 8 7 0`
   f. `0 4 6 5 3 8 1 7 2 9`
   g. `1 4 7 9 8 6 5 3 0 2`
   h. `2 1 4 3 6 5 8 7 9 0`

8. Suponga que se realiza una secuencia de operaciones de cola (`insertar` y `extraer`). Las operaciones de inserción colocan los enteros del 0 al 9, en orden, dentro de la cola; las operaciones extracción imprimen los valores que se extraen. ¿Cuál o cuáles de las siguientes secuencias no podrían ocurrir?

   a. `0 1 2 3 4 5 6 7 8 9`
   b. `4 6 8 7 5 3 2 9 0 1`
   c. `2 5 6 7 4 8 9 3 1 0`
   d. `4 3 2 1 0 5 6 7 8 9`

9. Escriba un algoritmo para encontrar el penúltimo nodo en una lista simplemente enlazada, en la cual el último nodo está indicado por una referencia `siguiente` igual a `None`.

10. Escriba un buen algoritmo para concatenar dos listas enlazadas simples `L` y `M`, dadas únicamente las referencias al primer nodo de cada lista, en una sola lista `L` que contenga todos los nodos de `L` seguidos por todos los nodos de `M`.

11. Escriba un algoritmo recursivo que cuente la cantidad de nodos en una lista simplemente enlazada.

12. Escriba un algoritmo para dos nodos `x` e `y` (no solo sus contenidos) en una lista simplemente enlazada `L`, teniendo únicamente las referencias a `x` e `y`. Repita el ejercicio para el caso en que `L` sea una lista doblemente enlazada. ¿Cuál de los dos algoritmos requiere más tiempo?

13. Suponga que `x` es un nodo de una lista enlazada y no es el último nodo de la lista. ¿Cuál es el efecto del siguiente fragmento de código?

```
x.siguiente = x.siguiente.siguiente
```

14. Suponga que x es un nodo de una lista enlazada e y es un nodo cualquiera. ¿Cuál es el efecto del siguiente fragmento de código?

```
y.siguiente = x.siguiente
x.siguiente = y
```

15. ¿Por qué el siguiente fragmento de código no hace la misma tarea que el fragmento del punto anterior?

```
x.siguiente = y
y.siguiente = x.siguiente
```